

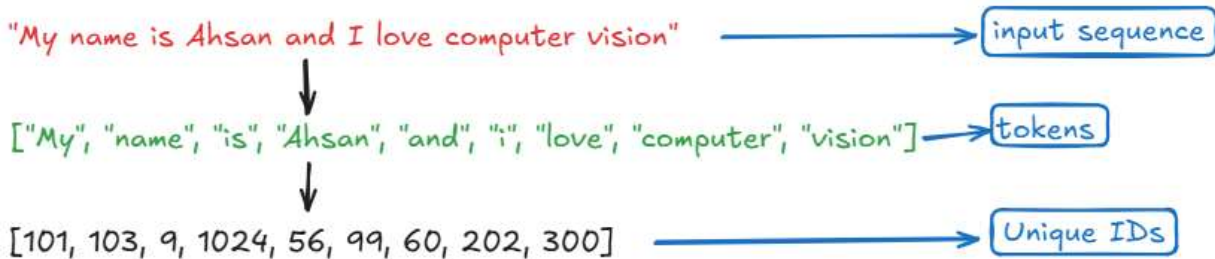
Training Tokenizer on custom dataset for LLMs



Why tokenizer?

LLM does not understand human-language, It only understand numbers. Therefore,

1. We split the input text/sequence into smaller part called **"tokens"**.
2. The tokenizer will assign a unique numerical IDs to each tokens.
3. These token IDs are then passed to LLM for processing.



Types of Tokenizer:

Two types of Tokenizer

1. Word-level tokenizer: Each word in the sequence is treated as token
2. Subword-level tokenizer: Each word in the sequence is divided into subwords, and each subword is treated as token.

Disadvantage of Word-level Tokenizer:

Problem with Word-level tokenizer:

1. Vocabulary Explosion: Each word in English grammar has many forms

such as: write, writes, written, writing etc.

Which means that there are billion of words in grammar,
So we need to treat each word as token.

Which requires large memory for LLM, training cost and time increases.
Also the LLM will not generalize well.

2. Out-of-Vocabulary Problem:

for example

Vocabulary [My, name, is, Ahsan, and, i, love, computer, vision, write, syntax, error, blogs]

Input "I am Ahsan and i writes blogs"

If tokenizer does not know the token, then it will assign unknown token to it

"writes" → <unk>

Sub-word Tokenizer:

Sub-word level Tokenizer:

Input My name is Ahsan and I writes blogs

Sub-word tokens ["wri", "te", "ting", "s", "write", "blogs", "my", "name", "i", "is", "Ahsan", "and"]

My name is Ahsan and i write s blogs